Problem 1

Problem. Download the PyDistMesh mesh generator. You can find all the necessary links from the Math 6660 Canvas page.

- Downloaded from Canvas. PyDistMesh is B. Froehle's Python port (2012) of P.-O. Persson's DistMesh (2004). Nothing in it was edited.

file	role
`distmesh2d.py`	`p,t = distmesh2d(fd,fh,h0,bbox,pfix)`: level set `fd`, relative size `fh`, edge length `h0`; returns nodes `p` ($N \times 2$) and triangles `t` ($M \times 3$)
`distance_functions.py`	level sets (`dcircle`, `drectangle`, ...) and Booleans: `dunion` = min, `dintersect` = max, `ddiff`$(d_1, d_2) = \max(d_1, -d_2)$
`boundary.py`	`bnd2d`: re-triangulates, marks nodes with $\lvert\phi\rvert \le 10^{-8}$ as boundary and puts them last, returns the areas
`assemble.py`	the cG(1) code used in Problem 3
`example1.py`, `example2.py`, `poisson_example.py`	unit disc; square with a hole, graded; $-\Delta u = 1$ on the latter

Table 1: The PyDistMesh bundle as downloaded (plus two helper files, `utils.py` and `mlcompat.py`).

Step 1 — what in the code is the lecture [Lecture 13, Part IV, eqs. (1.278)–(1.282)]

- `fd` is the lecture's level set ϕ: $\partial\Omega = \{\phi = 0\}$, $\Omega = \{\phi < 0\}$. The Booleans are the lecture's min (union) and max (intersection); a set difference is $\max(\phi_1, -\phi_2)$.
- Points become triangles by the lecture's two Python lines: a Delaunay triangulation from `scipy.spatial`, then only the triangles whose centroid has $\phi < 0$ are kept.
- Areas are the lecture's $\frac{1}{2}|\underline{v}_1 \times \underline{v}_2|$. Boundary nodes go last, so $u = 0$ is imposed by assembling only the first `num_int_nodes` rows and columns.

NOT FROM LECTURE — where the nodes come from

The lecture triangulates a given point set. DistMesh generates the points: a lattice of spacing h_0 cut to $\phi < 0$, then relaxed by treating every Delaunay edge as a compressed bar of rest length 1.2 `fh`, re-triangulating as nodes move and projecting escaped nodes back onto $\phi = 0$. Persson and Strang, *SIAM Review* 46 (2004).

Step 2 — it runs

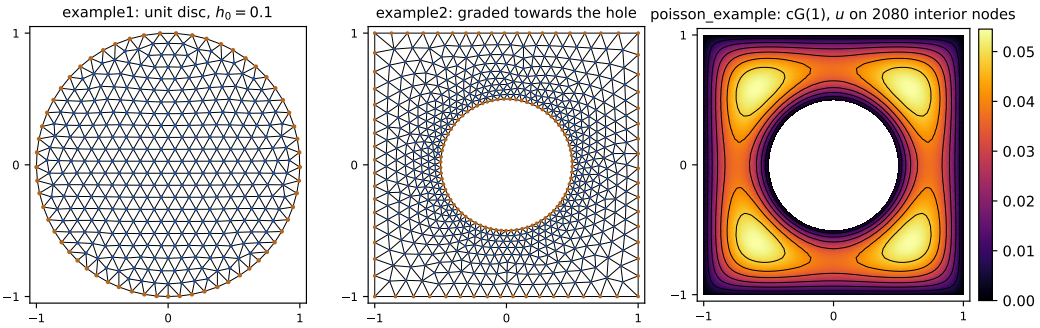


Figure 1: The three bundled examples, run unchanged. Interior nodes blue, boundary nodes brown. Right: the cG(1) solution of $-\Delta u = 1$, $u = 0$ on $\partial\Omega$.

example	h_0	interior	boundary	elements	mesh area	exact	$\max_j h_j/\rho_j$
unit disc, uniform	0.10	301	61	661	3.1359	3.1416	2.52
square with hole, graded	0.04	532	142	1206	3.2156	3.2146	2.83
square with hole, uniform, Poisson	0.04	2080	291	4451	3.2155	3.2146	2.64

Table 2: The three runs. Exact area: π for the disc, $4-\pi/4$ for the square with a hole of radius $1/2$. Last column: the lecture's regularity ratio (1.260), longest edge over inscribed diameter; an equilateral triangle gives $\sqrt{3}=1.73$.

- Areas agree to 0.2%; the disc comes out smaller and the holed square larger, since chords cut a sliver off a convex boundary and add one at a concave one. $\max_j h_j/\rho_j \le 2.83$: no slivers.

Problem 2

Problem. Use this code to generate a mesh for a star-shaped region whose boundary is the zero contour of $\phi(r,\theta) = r - 0.75 - 0.25\sin(5\theta)$.

Step 1 — the level set in x, y [Lecture 13, eqs. (1.279)–(1.280)]

- `fd` receives Cartesian points, so with $r=\sqrt{x^2+y^2}$, $\theta=\operatorname{atan2}(y,x)$:

$$\begin{gathered}\phi(x,y) = \sqrt{x^2+y^2} - 0.75 - 0.25\sin\big(5\,\operatorname{atan2}(y,x)\big),\\ \phi<0 \iff r < R(\theta) := 0.75 + 0.25\sin 5\theta.\end{gathered} \tag{2.1}$$

  $0.5 \le R \le 1$: five tips at $r=1$, five valleys at $r=0.5$; the box $(-1,1)^2$ contains Ω.

```
fd = lambda p: np.sqrt(p[:,0]**2 + p[:,1]**2) - 0.75 \
               - 0.25*np.sin(5*np.arctan2(p[:,1], p[:,0]))
p, t = dm.distmesh2d(fd, distfunc.huniform, h0, (-1,-1,1,1))
p, t, nin, nbd, area = bnd.bnd2d(fd, p, t, -1.0, -1.0, 1.0, 1.0)
```

- Exact area, to check the mesh against:

$$|\Omega| = \int_0^{2\pi}\int_0^{R(\theta)} r\,\mathrm{d}r\,\mathrm{d}\theta = \frac{1}{2}\int_0^{2\pi}\Big[0.75^2 + 1.5\cdot 0.25\underbrace{\sin 5\theta}_{\int=0} + 0.25^2\underbrace{\sin^2 5\theta}_{\int=\pi}\Big]\mathrm{d}\theta = \frac{19\pi}{32} = 1.86532. \tag{2.2}$$

Step 2 — the boundary nodes, and a fix

- `bnd2d` calls a node *boundary* only if $|\phi| \le 10^{-8}$, and Problem 3's solver sets $u=0$ only there. So every node on a boundary edge (an edge of one triangle only) must pass that test.
- **It does not.** At $h_0 = 0.05$ only 72 of the 138 nodes on boundary edges pass (Table 3, last column). The 66 others sit just inside, $-1.0\times10^{-4} \le \phi < -10^{-8}$, all on the inner part of the boundary, $r \le 0.78$, around the five valleys (Figure 2, red).
- **Why.** `distmesh2d` moves a node that left Ω back by $\boldsymbol{p} \leftarrow \boldsymbol{p} - \phi\nabla\phi/|\nabla\phi|^2$, repeated *while* $\phi > 10^{-14}$. One step lands exactly on $\phi=0$ only if ϕ is linear along $\nabla\phi$, as for $\phi = r-1$ in the bundle's examples. Here

$$\nabla\phi = \underbrace{\boldsymbol{e}_r}_{\partial\phi/\partial r} - \underbrace{\frac{1.25\cos 5\theta}{r}\,\boldsymbol{e}_\theta}_{\frac{1}{r}\partial\phi/\partial\theta} \tag{2.3}$$

  is not radial, so a step can land inside, and then the loop stops with $\phi<0$.

- **Fix** (the bundle is not edited). Before `bnd2d`, move every node on a boundary edge radially onto the curve:

$$(x, y) \;\leftarrow\; R(\theta)\,(\cos\theta, \sin\theta), \qquad \text{so} \quad \phi = R(\theta) - R(\theta) = 0. \tag{2.4}$$

  The nodes move at most $10^{-4} \ll h_0$. Afterwards every boundary node passes (Table 3).

Step 3 — the mesh [Lecture 13, eq. (1.260): regular triangulation]

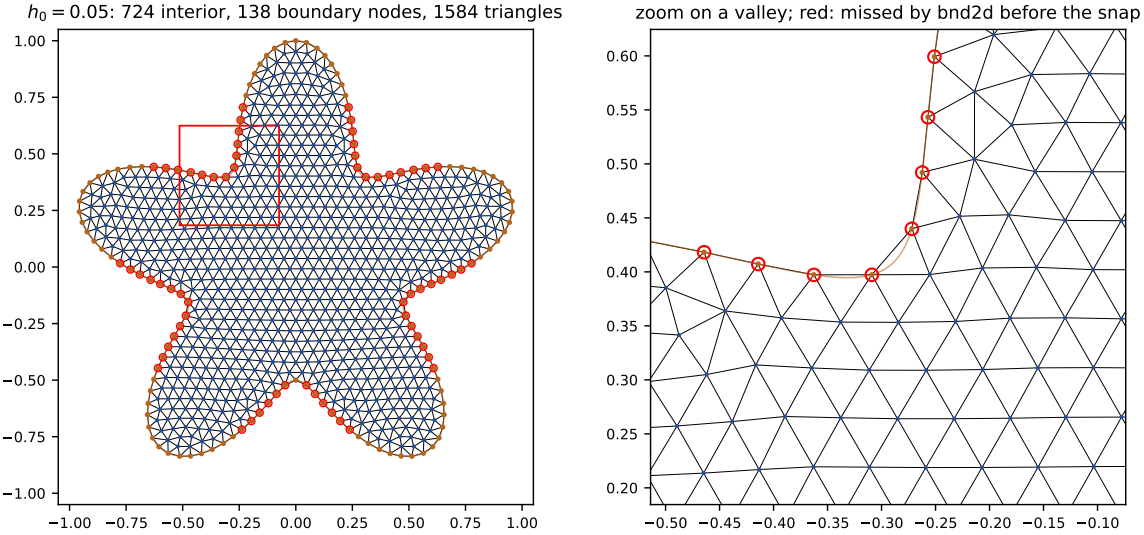


Figure 2: The star mesh at $h_0 = 0.05$, after the fix (2.4). Blue: interior nodes; brown: boundary nodes; the curve is $\phi = 0$. Red circles: the boundary nodes that `bnd2d` classed as interior before the fix.

h_0	interior	boundary	elements	mesh area	rel. error	$\max_j h_j/\rho_j$	flagged: raw → snapped
0.100	149	66	362	1.86001	-2.8×10^{-3}	2.57	32/66 → 66/66
0.050	724	138	1584	1.86432	-5.4×10^{-4}	2.46	72/138 → 138/138
0.025	3167	282	6614	1.86498	-1.9×10^{-4}	2.45	140/282 → 282/282

Table 3: Star meshes. Exact area $19\pi/32 = 1.86532$ (2.2). Last column: nodes `bnd2d` marks as boundary, out of the nodes on boundary edges, before and after (2.4).

- The area error falls from 0.3% to 0.02% as h_0 goes from 0.1 to 0.025.
- $\max_j h_j/\rho_j \le 2.57$ on every mesh (equilateral: 1.73): regular in the sense of (1.260), no slivers in the valleys.

Problem 3

Problem. Use the CG(1) method developed in class and provided on the Math 6660 Canvas page to solve $-u_{,xx} - u_{,yy} = 1$ in Ω, $u = 0$ on $\partial\Omega$, on the star of Problem 2.

Step 1 — the method [Lecture 13, eqs. (1.236) and (1.253)–(1.255)]

- Weak form: find $u \in H^1_0(\Omega)$ with $\iint_\Omega \nabla u \cdot \nabla v \,\mathrm{d}x\,\mathrm{d}y = \iint_\Omega f v \,\mathrm{d}x\,\mathrm{d}y$ for all v, $f = 1$.
- V_h: continuous, linear on each triangle of Problem 2, zero at the boundary nodes. Basis: the hat φ_i of each interior node. $U = \sum_i \xi_i \varphi_i$ and $\boldsymbol{A\xi} = \boldsymbol{b}$ with $A_{ij} = \iint \nabla\varphi_i \cdot \nabla\varphi_j$, $b_j = \iint f\varphi_j$.

Step 2 — what `assemble.py` computes, triangle by triangle

- On a triangle K with vertices (x_l, y_l), $l = 1, 2, 3$, the hat of vertex k is $a_k + b_k x + c_k y$ with value δ_{kl} at vertex l:

$$\underbrace{\begin{bmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{bmatrix}}_{\boldsymbol{M}} \begin{bmatrix} a_k \\ b_k \\ c_k \end{bmatrix} = \boldsymbol{e}_k \quad \Longrightarrow \quad (a_k, b_k, c_k)^T = \text{column } k \text{ of } \boldsymbol{M}^{-1}, \tag{3.1}$$

  and $\nabla\varphi_k = (b_k, c_k)$ on K.

- The gradients are constant on K, so the stiffness integral is exact:

$$\iint_K \nabla\varphi_j \cdot \nabla\varphi_k \,\mathrm{d}x\,\mathrm{d}y = |K|\,(b_j b_k + c_j c_k). \tag{3.2}$$

- The load uses the vertex rule, Lecture 4's trapezoidal rule on a triangle:

$$\iint_K f\varphi_k \,\mathrm{d}x\,\mathrm{d}y \approx \frac{|K|}{3} \sum_{l=1}^{3} f(x_l, y_l)\, \underbrace{\varphi_k(x_l, y_l)}_{\delta_{kl}} = \frac{|K|}{3}\, f(x_k, y_k). \tag{3.3}$$

  For $f \equiv 1$ it is exact. φ_k is linear, so its mean over K is its value at the centroid, the average of the vertex values, $(1 + 0 + 0)/3$.

- Only nodes with index below `num_int_nodes` get a row and a column: $u = 0$ is built into V_h. The assembled $\boldsymbol{A}$ is symmetric to round-off ($\max|\boldsymbol{A} - \boldsymbol{A}^T| = 0$).

Step 3 — check on a domain with a known answer

- Unit disc: $u = \frac{1}{4}(1 - x^2 - y^2)$ has $u_{,xx} = u_{,yy} = -\frac{1}{2}$, so $-\Delta u = 1$, and $u = 0$ at $r = 1$. Same pipeline, no fix needed: for $\phi = r - 1$ every boundary node passes.

h_0	interior nodes	$\max U$	$\max_i \lvert U_i - u(x_i, y_i)\rvert$	rate
0.100	301	0.24983	4.03×10^{-4}	—
0.050	1324	0.24984	9.63×10^{-5}	2.06
0.025	5559	0.25000	2.37×10^{-5}	2.02

Table 4: The Canvas cG(1) code on the unit disc, $-\Delta u = 1$: largest nodal error against $\frac{1}{4}(1 - x^2 - y^2)$. It falls as h_0^2.

Step 4 — the star

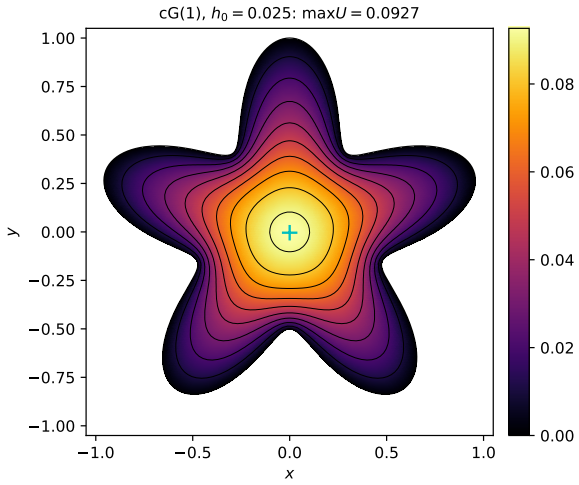


Figure 3: cG(1) solution of $-\Delta u = 1$, $u = 0$ on the star, $h_0 = 0.025$ (3167 unknowns), with ten contours. Cross: the largest nodal value.

h_0	unknowns	$\max U$	at (x, y)	$\iint_{\Omega_h} U\,dx\,dy$	$\max U$, raw mesh
0.100	149	0.09233	(0.000, 0.032)	0.06619	0.23841
0.050	724	0.09256	(−0.026, −0.008)	0.06747	0.22573
0.025	3167	0.09266	(0.000, −0.004)	0.06764	0.22951

Table 5: The star under refinement. "Raw mesh": the same nodes without the fix (2.4) of Problem 2.

- $\max U = 0.0927$, at the node nearest the centre — where the five-fold symmetry puts it — and stable to three digits from $h_0 = 0.05$.
- Without the fix, $\max U \approx 0.23$: the nodes missed in Problem 2 are left free, so along the valleys the code imposes no condition instead of $u = 0$. The solution is then wrong.

Result. On the star, $\max u \approx 0.0927$ at the centre and $\iint_\Omega u\,\mathrm{d}x\,\mathrm{d}y \approx 0.0676$ (Table 5, Figure 3).

Mechanics reading. A membrane of unit tension under unit pressure, rim clamped. The disc inscribed in the star ($r = 0.5$) deflects $1/16 = 0.0625$ at its centre, the circumscribed disc ($r = 1$) $1/4$; the star sits between, at 0.0927. The raw-mesh 0.23 is almost the circumscribed value: a membrane released along its valleys.

Problem 4(d)

Problem. Use the method of 4(c) to solve $-\nabla\cdot\nabla u = f$ on $\Omega=[-1,1]^2$, $u=g$ on $\partial\Omega$, with

$$f=\pi^2 e^{\sin(\pi x)\sin(\pi y)}\big\{2\sin(\pi x)\sin(\pi y)+2\cos^2(\pi x)\cos^2(\pi y)-\cos^2(\pi x)-\cos^2(\pi y)\big\},\qquad g=1.$$

Create a convergence table showing both $\|\cdot\|_V$ and $\|\cdot\|_{L^2(\Omega)}$ errors. What order of convergence do you observe? The exact solution is $u=e^{\sin(\pi x)\sin(\pi y)}$.

Step 1 — the data are consistent

- With $s=\sin\pi x\sin\pi y$ and $u=e^s$, $u_{,x}=\pi\cos\pi x\sin\pi y\,e^s$ and

$$\begin{aligned}-u_{,xx}-u_{,yy}&=\pi^2e^s\big[2\sin\pi x\sin\pi y-\cos^2\pi x\ \underbrace{\sin^2\pi y}_{1-\cos^2\pi y}-\underbrace{\sin^2\pi x}_{1-\cos^2\pi x}\cos^2\pi y\big]\\&=\pi^2e^s\big[2\sin\pi x\sin\pi y+2\cos^2\pi x\cos^2\pi y-\cos^2\pi x-\cos^2\pi y\big]=f.\end{aligned}\tag{4d.1}$$

  Also checked symbolically ($-\Delta u-f\equiv 0$).
- On every side $\sin(\pm\pi)=0$, so $s=0$ and $u=e^0=1=g$.

Step 2 — the implementation [4(c); Lecture 5, eq. (1.66)]

- **Grid.** $N\times N$ squares, $N=4,\dots,256$, $h=2/N$, $(N-1)^2$ unknowns.
- **Stiffness.** The element matrix $\boldsymbol{k}^e$ of 4(c), the same on every square, assembled. Checked in the code: every interior row is the stencil $\frac{1}{3}[8;-1\times 8]$ of 4(c), the interior matrix is symmetric positive definite, and the boundary terms are 5/3, 1, 0 as in 4(c).
- **Boundary.** $U=g=1$ at boundary nodes, moved to the right-hand side (Lecture 5).
- **Load.** $b_{ij}=\iint f\varphi_{ij}$ by 3×3 Gauss points per square, not the trapezoidal h^2f_{ij} of 4(c); Step 4 compares.
- **Errors,** 4×4 Gauss points per square, with the norm of Lecture 13, eq. (1.254):

$$\|e\|^2_{L^2(\Omega)}=\iint_\Omega (u-U)^2\,\mathrm{d}x\,\mathrm{d}y,\qquad \|e\|^2_V=\|e\|^2_{L^2(\Omega)}+\iint_\Omega|\nabla(u-U)|^2\,\mathrm{d}x\,\mathrm{d}y.\tag{4d.2}$$

  Rate $=\log_2\big(E(h)/E(h/2)\big)$.

Step 3 — convergence table

N	h	unknowns	$\|e\|_V$	rate	$\|e\|_{L^2(\Omega)}$	rate
4	0.50000	9	2.1378×10^{0}	—	2.3519×10^{-1}	—
8	0.25000	49	1.4636×10^{0}	0.55	9.5737×10^{-2}	1.30
16	0.12500	225	7.3914×10^{-1}	0.99	2.4256×10^{-2}	1.98
32	0.06250	961	3.7048×10^{-1}	1.00	6.0808×10^{-3}	2.00
64	0.03125	3969	1.8536×10^{-1}	1.00	1.5213×10^{-3}	2.00
128	0.01562	16129	9.2693×10^{-2}	1.00	3.8038×10^{-4}	2.00
256	0.00781	65025	4.6348×10^{-2}	1.00	9.5100×10^{-5}	2.00

Table 6: Bilinear squares on $[-1,1]^2$, $N\times N$ elements.

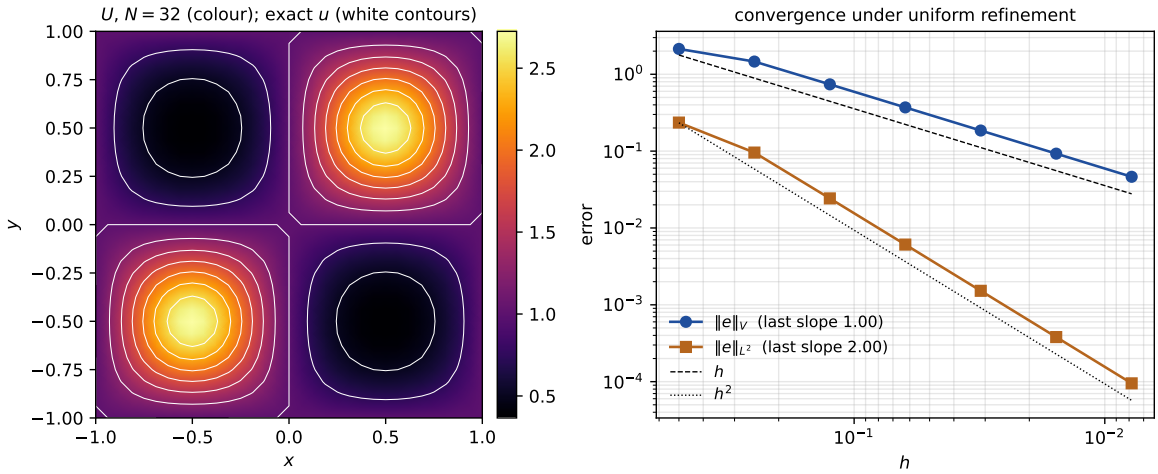


Figure 4: Left: U at $N = 32$ with contours of the exact u. Right: the errors of Table 6 against h, with slopes 1 and 2 for reference.

Step 4 — the observed orders

- $\|e\|_V$: rate $\to 1.00$, i.e. $\mathcal{O}(h)$. $\|e\|_{L^2(\Omega)}$: rate $\to 2.00$, i.e. $\mathcal{O}(h^2)$. Clean from $N = 16$; at $N = 4, 8$ the mesh has only two to four squares across each bump of u.
- The trapezoidal load of 4(c) gives the same rates; at $N = 256$, $\|e\|_V = 4.6355 \times 10^{-2}$ and $\|e\|_{L^2} = 9.7057 \times 10^{-5}$, against 4.6348×10^{-2} and 9.5100×10^{-5} with Gauss.
- **Why $\mathcal{O}(h)$ in $\|\cdot\|_V$.** $g = 1$ is reproduced exactly by the boundary terms of 4(c), so $U - 1$ is the Galerkin approximation of $w = u - 1 \in H^1_0$, and Céa's lemma (Lecture 13, eq. (1.242)) with the constants of 4(b) gives

$$\|u - U\|_V \le \frac{\beta^*}{\beta_*} \|u - v\|_V = 8 \|u - v\|_V \qquad \text{for every admissible } v \in V_h. \tag{4d.3}$$

  With v the bilinear interpolant of u, the right side is $\mathcal{O}(h)$.
- **Why $\mathcal{O}(h^2)$ in L^2.** Lecture 7's duality argument: the interpolation estimate is used twice, once on e and once on the dual solution, one factor of h each.

NOT FROM LECTURE — the interpolation estimate for squares

Lecture 13 states $\|u - u_h\|_{H^1} \le c \|h\, u\|_{H^2}$, eq. (1.262), for linear triangles. The same $\mathcal{O}(h)$ bound for the bilinear interpolant on squares, and the 2D duality argument behind $\mathcal{O}(h^2)$ in L^2, are standard but were not shown in class. Table 6 confirms both rates.

Conclusion. Bilinear squares converge at $\mathcal{O}(h)$ in $\|\cdot\|_V$ and $\mathcal{O}(h^2)$ in $\|\cdot\|_{L^2(\Omega)}$ — the same orders as CG(1) — with $\|e\|_V = 4.63 \times 10^{-2}$ and $\|e\|_{L^2} = 9.51 \times 10^{-5}$ at $N = 256$.